

MINI SCIENTIFIC CALCULATOR

Technical Documentation

Written in TASM x86 16-bit Assembly

Supports: Addition · Subtraction · Multiplication · Division
Exponent · Percentage · Factorial

Group of Choa (2ITB)

- Choa, Theodore Kyle
 - Del Rosario, Carlos
 - Elcana, Joaquin
- Evaristo, Ron Christian
 - Reyes, Kristian Alrich
 - Villamil, Rohan Viktor
 - Yan, Stevenson Telly

1. Introduction

This document describes the design, implementation, and behavior of the Mini Scientific Calculator — a command-line application written entirely in TASM (Turbo Assembler) x86 16-bit Assembly language and targeted at MS-DOS or DOSBox environments.

Most modern calculators are built in high-level languages that abstract away hardware details. This project takes the opposite approach: every arithmetic operation — from a simple addition to factorial computation — is implemented directly using CPU registers, the x86 instruction set, and DOS interrupt calls. There are no standard libraries and no runtime support. What you see in the source file is what the processor executes, instruction by instruction.

1.1 Motivation & Context

Assembly language programming is the closest a software developer can get to the hardware without writing machine code by hand. By building a calculator in TASM, this project demonstrates:

- How integers are stored and manipulated at the register level
- How 32-bit arithmetic is performed on a 16-bit processor using register pairs (DX:AX)
- How DOS services (INT 21h) are used for keyboard input and screen output
- How control flow — branches, loops, and procedure calls — is expressed with jump and call instructions
- How signed numbers are handled manually through sign-flag bookkeeping, without relying on CPU overflow flags for complex cases

1.2 Feature Summary

Operation	Symbol / Trigger
Addition	+ entered as operator
Subtraction	- entered as operator
Multiplication	* entered as operator
Division	/ entered as operator
Exponentiation	^ entered as operator
Percentage of	% entered as operator
Factorial	! entered as operator (one operand only)
Exit	Type EXIT at the first number prompt

1.3 Numeric Range

All arithmetic uses 32-bit signed integers stored in two 16-bit word pairs. The effective range for inputs is 0 to 2,147,483,647 (positive only for factorial and percentage; signed for add/sub/mul)

Inputs outside this range produce undefined or wrapped results because no overflow detection is implemented beyond what is inherent in the register pair structure.

1.4 Build & Run

The program is assembled and linked with Turbo Assembler (TASM) and Turbo Linker (TLINK), tools from Borland that target the DOS real-mode environment. To assemble and run:

```
TASM SCICALC.ASM      ; Assemble → SCICALC.OBJ
TLINK SCICALC.OBJ     ; Link    → SCICALC.EXE
SCICALC.EXE          ; Run inside DOS / DOSBox
```

2. Code Walkthrough

The following sections examine the most architecturally significant parts of the source code and explain what each instruction accomplishes and why.

2.1 Memory Model and Segment Setup

```
.MODEL SMALL
.STACK 400h

.DATA
    N1L    DW 0    ; num1 low  word
    N1H    DW 0    ; num1 high word
    N2L    DW 0
    N2H    DW 0
    RL     DW 0    ; result low
    RH     DW 0    ; result high

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
```

The SMALL memory model places code and data in separate 64 KB segments, which is sufficient for a DOS utility. The .STACK 400h directive reserves 1024 bytes (0x400) of stack space — enough to handle nested CALL/PUSH operations without overflow.

The two lines MOV AX, @DATA / MOV DS, AX are the standard DOS segment initialization idiom. The assembler resolves @DATA to the physical segment address of the .DATA section. Because the processor starts with DS pointing to an undefined location, loading it explicitly is mandatory before any data variables can be read or written. Without this, every memory reference to N1L, N2L, RL, and so on would produce garbage values.

Each 32-bit number is split into two 16-bit variables — a Low word (N1L) and a High word (N1H). This pattern is required because the 8086/8088 family used by DOS is a 16-bit architecture, with registers only 16 bits wide. The DX:AX register pair mimics the same low/high layout during arithmetic.

2.2 DOS Buffered Input (INT 21h / 0Ah)

```
inbuf  DB 20, 0, 20 DUP(0)    ; [max_len][actual_len][data...]

READ_LINE PROC
    LEA  DX, inbuf
    MOV  AH, 0Ah
    INT  21h                ; DOS buffered keyboard read
    ; emit CR + LF manually
    MOV  DL, 13
    MOV  AH, 02h
    INT  21h
    MOV  DL, 10
    MOV  AH, 02h
    INT  21h
    RET
READ_LINE ENDP
```

DOS interrupt 21h with AH=0Ah performs buffered keyboard input. The buffer layout is fixed by DOS: the first byte is the maximum number of characters the buffer can hold (20), the second byte is filled by DOS with the

actual number of characters typed, and the remaining bytes hold the input string terminated by a carriage-return (0Dh).

The program then emits a carriage return (ASCII 13) followed by a line feed (ASCII 10) using INT 21h / AH=02h to advance the cursor to a new line. DOS buffered input does not automatically advance the cursor, so without these two explicit calls, the next prompt would overwrite the line just typed.

The data is accessed later as inbuf+2 — skipping the two control bytes — to reach the raw characters the user entered.

2.3 Case-Insensitive EXIT Detection (CHK_EXIT)

```
CHK_EXIT PROC
    MOV     EXITF, 0
    LEA     SI, inbuf+2
    MOV     AL, [SI]
    AND     AL, 0DFh        ; force uppercase (clear bit 5)
    CMP     AL, 'E'
    JNE     CX_NO
    ; repeat for X, I, T ...
    MOV     EXITF, 1
CX_NO:
    RET
CHK_EXIT ENDP
```

The trick AND AL, 0DFh deserves special attention. In ASCII, lowercase letters differ from their uppercase counterparts only in bit 5 (value 0x20). For example, 'e' is 0x65 and 'E' is 0x45 — the only difference is bit 5. Clearing bit 5 with the mask 0xDF (binary 1101 1111) maps any lowercase letter to its uppercase equivalent without a branch or a lookup table. This lets the user type exit, EXIT, Exit, or any capitalisation and the program handles it correctly.

The function checks each character of the word EXIT sequentially. If any character does not match, it jumps to CX_NO and leaves EXITF at 0. Only when all four match is EXITF set to 1, which causes the main loop to jump to L_DONE and terminate.

2.4 32-bit Integer Parsing (PARSE32)

```
PARSE32 PROC
    LEA     SI, inbuf+2
    MOV     N1NEG, 0
    MOV     BX, 0           ; accumulator low
    MOV     DI, 0           ; accumulator high
    MOV     AL, [SI]
    CMP     AL, '-'
    JNE     P32_1_NEXT
    MOV     N1NEG, 1        ; record negative sign
    INC     SI
P32_1_NEXT:
    ; for each digit: acc = acc * 10 + digit
    MOV     AX, BX
    MUL     CX              ; DX:AX = BX * 10
    MOV     BX, AX
    ; ... add high-word carry, then add digit
    ADD     BX, AX
    ADC     DI, 0
P32_1_DONE:
    MOV     N1L, BX
    MOV     N1H, DI
    RET
PARSE32 ENDP
```

PARSE32 converts the raw ASCII string in the input buffer into a 32-bit binary integer stored across N1L (low 16 bits) and N1H (high 16 bits). It implements the standard "multiply-accumulate" algorithm: for each digit character, the running total is multiplied by 10 and the new digit is added.

Because the accumulator spans two 16-bit registers (DI:BX), the multiplication must be done in two steps. BX is multiplied by 10 first, producing a 32-bit result in DX:AX. The carry in DX is preserved and added to DI*10. This correctly propagates carries from the low word into the high word, so numbers larger than 65535 parse without truncation.

The sign flag N1NEG is stored separately rather than using two's-complement, which is a deliberate design choice. It simplifies the addition and subtraction logic by eliminating the need to compare signs and choose the appropriate operation.

2.5 Signed 32-bit Addition (DO_ADD)

```
DO_ADD:
    MOV     AL, N1NEG
    CMP     AL, N2NEG
    JNE     ADD_DIFF_SIGN

    ; Same sign: add magnitudes, keep sign
    MOV     AX, N1L
    MOV     DX, N1H
    ADD     AX, N2L           ; add low words
    ADC     DX, N2H           ; add high words + carry
    MOV     RL, AX
    MOV     RH, DX
    MOV     AL, N1NEG
    MOV     RNEG, AL
    JMP     SHOW_RESULT

ADD_DIFF_SIGN:
    ; Compare magnitudes, subtract smaller from larger
    ; result sign = sign of the larger operand
```

This is a sign-magnitude addition, not two's-complement. The processor instructions ADD and ADC (Add with Carry) handle the 32-bit binary sum in two 16-bit steps: ADD adds the low words and sets the carry flag if there is a carry-out; ADC then adds the high words and also adds that carry flag, correctly propagating the carry across the word boundary.

When the two operands have different signs, the operation becomes a subtraction of magnitudes. The code first compares N1H against N2H (the higher word determines the larger number, unless they are equal). If the high words are equal, it falls through to comparing the low words. This two-level comparison correctly determines which of the 32-bit magnitudes is larger before subtraction. The result takes the sign of the larger operand — exactly the rule for signed arithmetic.

2.6 32-bit Multiplication (DO_MUL)

```
DO_MUL:
    ; Step 1: N1L * N2L -> DX:AX (bits 0-31)
    MOV     AX, N1L
    MUL     N2L
    MOV     RL, AX
    MOV     RH, DX

    ; Step 2: N1L * N2H -> AX (bits 16-31 contribution)
    MOV     AX, N1L
```

```

MUL  N2H
ADD  RH, AX

; Step 3: N1H * N2L -> AX (bits 16-31 contribution)
MOV  AX, N1H
MUL  N2L
ADD  RH, AX

```

The x86 MUL instruction multiplies AX by its operand and places the full 32-bit result in DX:AX. To multiply two 32-bit numbers and keep the low 32 bits of the result, the operation is split into partial products, mirroring long multiplication by hand but in base 65536.

Representing N1 as $(N1H * 65536 + N1L)$ and N2 as $(N2H * 65536 + N2L)$, the full product's low 32 bits come from three terms: $N1L * N2L$ (contributes to bits 0-31), $N1L * N2H$ shifted left 16 (bits 16-31), and $N1H * N2L$ shifted left 16 (bits 16-31). The fourth term $N1H * N2H$ would land in bits 32-63 and is discarded, which is valid as long as the true product fits in 32 bits. The result sign follows the XOR of the two input signs — negative only when exactly one input is negative.

2.7 32-bit Division (DO_DIV)

```

DO_DIV:
    MOV  AX, N2L
    OR   AX, N2H
    JNZ  DIV_OK
    ; Print error and skip

DIV_OK:
    ; Step 1: high word / divisor
    MOV  AX, N1H
    MOV  DX, 0
    DIV  N2L          ; AX = high quotient, DX = remainder
    MOV  RH, AX
    ; Step 2: remainder:N1L / divisor
    MOV  AX, N1L
    DIV  N2L          ; AX = low quotient
    MOV  RL, AX

```

The x86 DIV instruction divides the DX:AX register pair by its operand, yielding AX=quotient and DX=remainder. To divide a 32-bit dividend by a 16-bit divisor in two steps, the algorithm first divides the high word (N1H) with DX cleared, obtaining a high-quotient and a remainder. That remainder becomes the new DX for the second DIV, which divides remainder:N1L by the same divisor to produce the low quotient.

The zero-check uses `OR AX, N2H` — ORing both halves of the divisor together. If the divisor is zero, both words are zero, so the OR result is zero and JNZ falls through to the error path. This is more compact than two separate CMP instructions and is a common assembly idiom for testing whether a 32-bit value is zero.

A current limitation is that the divisor is assumed to fit in 16 bits (N2L only). Divisors larger than 65535 are silently truncated. For the typical use-cases this calculator is designed for, this is acceptable.

2.8 Exponentiation (DO_POW)

```

DO_POW:
    MOV  CX, N2L          ; exponent in CX
    JCXZ POW_ZERO        ; if exponent = 0, result = 1
    MOV  BX, 1            ; accumulator lo
    MOV  BP, 0            ; accumulator hi
POW_LOOP:
    ; acc(BP:BX) = acc * N1 [32x32->32 partial products]
    MOV  AX, BX

```

```

MUL  N1L
MOV  DI, AX
PUSH DX
; ... (same partial product pattern as DO_MUL)
LOOP POW_LOOP

```

Exponentiation is computed iteratively: starting with an accumulator of 1, the base (N1) is multiplied in CX times, where CX holds the exponent. The LOOP instruction decrements CX and jumps back if CX is not zero — it is the canonical x86 counted loop instruction.

Inside the loop, the 32-bit accumulator is kept in the BP:BX register pair (not DX:AX, which is consumed by MUL). The multiplication uses the same three-partial-product technique from DO_MUL. The JCXZ (Jump if CX is Zero) at the start handles the special case of a zero exponent before entering the loop, since zero iterations of the LOOP instruction would otherwise require CX to wrap around 65536 times.

2.9 Factorial (DO_FACT)

```

DO_FACT:
    CMP  N1NEG, 1
    JNE  FACT_NOTNEG
    ; Print error: factorial of negative
FACT_NOTNEG:
    ; 0! = 1 special case
    MOV  CX, N1L      ; loop from 2 to N
    MOV  DI, 2        ; current multiplier
FACT_LOOP:
    CMP  DI, CX
    JA   FACT_DONE
    MOV  AX, BX
    MUL  DI            ; acc_lo * DI
    ; ... propagate carry into BP
    INC  DI
    JMP  FACT_LOOP

```

Factorial is computed as a running product: acc starts at 1, then is multiplied by 2, 3, 4, ... up to N. A separate multiplier register DI counts up from 2, and the loop continues while DI is less than or equal to CX (which holds N). This avoids the LOOP instruction and instead uses a manual CMP/JA pair, which is necessary because the multiplier is not the loop counter.

Two guard conditions are checked before entering the loop: if N1NEG is 1 (a negative number was entered) an error message is shown, since factorial is undefined for negative integers. If N1 is zero, the result is immediately set to 1 because $0! = 1$ by mathematical convention, and the loop is skipped entirely.

2.10 32-bit Decimal Output (PRINT32)

```

PRINT32 PROC
    ; Push digits onto stack, then pop and print
    MOV  CX, 0        ; digit counter
PR32_DIV:
    MOV  AX, SI        ; high word
    MOV  DX, 0
    DIV  BX            ; BX = 10
    MOV  SI, AX        ; new high
    MOV  AX, DI        ; low word
    DIV  BX            ; AX = new low, DX = digit
    ADD  DL, '0'       ; convert to ASCII
    PUSH DX
    INC  CX
    JMP  PR32_DIV
PR32_PRINT:

```

```
POP    DX
MOV    AH, 02h
INT     21h      ; print character in DL
LOOP   PR32_PRINT
PRINT32 ENDP
```

Converting a binary integer to a decimal string requires extracting digits from least-significant to most-significant (by repeated division by 10), but they must be printed from most-significant to least-significant. The stack solves this naturally: each extracted digit is pushed, and after all digits are extracted they are popped and printed in reverse order.

Each digit is converted from a binary value (0-9) to its ASCII character by adding the ASCII code of '0' (48). INT 21h / AH=02h prints a single character from DL — the standard DOS character-output service. CX tracks how many digits were pushed so LOOP knows when to stop popping.

3. Program Architecture Overview

3.1 Data Segment Layout

Variable	Purpose
banner	Welcome screen string (DB, \$-terminated for INT 21h/09h)
pr1, pr2, prop	Prompt strings for first number, operator, second number
pr_res	Result label string
pr_sep	Separator line printed after each calculation
pr_bye	Goodbye message shown on EXIT
pr_ediv	Error string: Division by zero
preneg	Error string: Factorial of a negative number
pr_eop	Error string: Unknown operator symbol
pr_minus	Minus sign string, printed before negative results
inbuf	20-byte DOS buffered input buffer [max][actual][data...]
N1L / N1H	32-bit first operand as two 16-bit words
N2L / N2H	32-bit second operand as two 16-bit words
RL / RH	32-bit result as two 16-bit words
N1NEG / N2NEG	Sign flags: 1=negative, 0=positive
RNEG	Result sign flag
OP	Single-byte operator character (+,-,*,/,^,%,!)
EXITF	Exit flag: 1 when user types EXIT

3.2 Execution Flow

The program follows a simple read-evaluate-print loop (REPL) common to interpreters and calculators:

1. Display welcome banner
2. Prompt for the first number → READ_LINE → CHK_EXIT → PARSE32
3. If EXIT detected, jump to goodbye message and terminate
4. Prompt for operator → READ_LINE → store in OP
5. If operator is '!', skip to DO_FACT (no second operand needed)
6. Prompt for second number → READ_LINE → PARSE32_2
7. Dispatch to the appropriate operation based on OP
8. Print result via SHOW_RESULT → PRINT32
9. Print separator line → jump back to step 2

3.3 Procedure Summary

Procedure	Role
MAIN	Entry point; hosts the main REPL loop and operation dispatch
READ_LINE	Reads one line from the keyboard using DOS INT 21h/0Ah
CHK_EXIT	Checks if the input is the word EXIT (case-insensitive)

PARSE32	Parses input string into 32-bit integer → N1L, N1H, N1NEG
PARSE32_2	Same as PARSE32 but targets N2L, N2H, N2NEG
PRINT32	Prints a 32-bit unsigned value in DX:AX as decimal digits

4. Debugging — Example Equations and Expected Results

The following table lists representative test cases that exercise each operation and error path. Run these inputs in the calculator to verify correct behavior.

4.1 Standard Test Cases

Operation	First Number	Operator	Second Number	Expected Result
Addition	250	+	750	1000
Addition (neg)	-300	+	500	200
Addition (neg)	-400	+	-100	-500
Subtraction	1000	-	250	750
Subtraction	50	-	200	-150
Multiplication	123	*	456	56088
Multiplication	-12	*	5	-60
Division	100	/	4	25
Division	7	/	2	3 (integer, truncated)
Exponent	2	^	10	1024
Exponent	3	^	0	1
Percentage	20	%	500	100
Percentage	15	%	200	30
Factorial	5	!	(none)	120
Factorial	0	!	(none)	1
Factorial	10	!	(none)	3628800

4.2 Error and Edge Cases

Scenario	First Number	Operator	Second Number	Expected Output
Divide by zero	100	/	0	ERROR: Division by zero!
Factorial of negative	-5	!	(none)	ERROR: Factorial of negative!
Unknown operator	10	@	5	ERROR: Unknown operator!
Zero exponent	999	^	0	1
Zero factorial	0	!	(none)	1
Add opposites	500	+	-500	0
Exit command	EXIT	-	-	Goodbye! (program ends)
Case-insensitive exit	exit	-	-	Goodbye! (program ends)

4.3 Large Number Tests

These tests exercise the 32-bit arithmetic paths:

Operation	First Number	Second Number	Expected Result
Addition (large)	2000000000	147483647	2147483647 (max signed 32-bit)
Multiplication (medium)	65535	65535	4294836225
Factorial (12)	12	(none)	479001600
Factorial (13)	13	(none)	1932053504 (overflow — wraps)
Power 2^30	2	1073741824 (truncated — overflow)	1073741824

Note: 13! = 6,227,020,800 exceeds 32-bit capacity (max 4,294,967,295). The result wraps around silently. This is expected behaviour given the 32-bit integer constraint.

5. Known Limitations and Constraints

5.1 Arithmetic Limitations

- Integer only — no floating-point support. Division truncates toward zero.
- No overflow detection — results silently wrap for values exceeding 2,147,483,647.
- Division assumes the divisor fits in 16 bits ($\leq 65,535$). Larger divisors are truncated to N2L.
- Percentage uses only N1L * N2L (low words only), so both inputs should be $\leq 65,535$ for correct results.
- Factorial overflows for inputs ≥ 13 ($13! = 6.2$ billion $>$ 32-bit max).
- Exponent overflows quickly; a practical safe range is base $\leq 46,340$ with exponent 2.

5.2 Input Limitations

- Negative numbers are supported only for addition and subtraction; the factorial and percentage routines ignore or return an error for negative inputs.
- Maximum input length is 20 characters (set in inbuf). Longer input is silently truncated.
- No input validation beyond digit checking — non-digit, non-minus characters terminate parsing silently.
- Only single-character operators are recognized; multi-character operator names (e.g., "mod") are not supported.

5.3 Environment Limitations

- Requires a real-mode DOS environment (MS-DOS, FreeDOS, or DOSBox). Will not run natively on 64-bit Windows or Linux without an emulator.
- Assembled with TASM/TLINK (Borland toolchain). MASM-compatible syntax differences may require minor edits if porting.

6. Potential Enhancements

The following improvements could extend the calculator while staying within the TASM/DOS environment:

6.1 Arithmetic Improvements

- Floating-point support using the x87 FPU instructions (FILD, FDIV, FSTP) for real division and decimal results.
- 64-bit integer arithmetic by extending register pairs to four 16-bit words per operand.
- Overflow detection using the CPU Carry and Overflow flags after each arithmetic operation.
- Modulo operation (the remainder from integer division, already computed in DX after DIV).
- Square root using Newton-Raphson iteration in integer arithmetic.

6.2 Input / Output Improvements

- Decimal point input and fixed-point arithmetic for more natural everyday calculations.
- History — store the last N results and allow them to be recalled as operands.
- Expression parsing — allow the user to type a full expression such as "3 + 4 * 2" on one line.
- Input editing — allow backspace and cursor movement during entry (requires raw keyboard read via INT 16h instead of INT 21h/0Ah).

6.3 Robustness Improvements

- Overflow guard for factorial — detect when the accumulator exceeds 2,147,483,647 and display an overflow warning instead of a wrapped result.
- Full 32-bit divisor support in DO_DIV using a software long-division routine.
- Graceful handling of non-numeric input rather than silent truncation.

7. Register and Interrupt Reference

7.1 Register Roles

Register	Role in this program
AX	Primary arithmetic register; also holds DOS function codes in AH and character data in AL
BX	Low-word accumulator in factorial and power loops; also used as divisor (=10) in PRINT32
CX	Loop counter (LOOP instruction); also multiplier constant (=10) in PARSE32
DX	High-word in 32-bit MUL/DIV results; also used for INT 21h string pointer and character output
SI	Source index — pointer into inbuf during parsing; also high-word accumulator in PRINT32
DI	High-word accumulator during PARSE32; low-word source in PRINT32; multiplier in DO_FACT
BP	High-word accumulator in DO_POW and DO_FACT (preserved across CALL)
DS	Data segment register — initialised to @DATA at startup
SP/SS	Stack segment and pointer — managed automatically by PUSH/POP/CALL/RET

7.2 DOS Interrupts Used

INT / AH	Function
INT 21h / AH=09h	Print \$-terminated string; DX = offset of string
INT 21h / AH=0Ah	Buffered keyboard input; DX = offset of input buffer
INT 21h / AH=02h	Print single character in DL
INT 21h / AH=4Ch	Terminate program; AL = exit code (0 = success)

8. Conclusion

The Mini Scientific Calculator demonstrates that a complete, interactive arithmetic tool can be built entirely in x86 assembly language without any external libraries or runtime support. Every step — reading keyboard input, parsing decimal text into binary, performing multi-step 32-bit arithmetic, and converting results back to readable decimal output — is performed through direct register manipulation and a small set of DOS interrupt calls.

From an educational perspective, the program illuminates several foundational concepts that high-level languages routinely hide: why 32-bit numbers require two passes through a 16-bit MUL or DIV instruction, why division by zero must be caught explicitly rather than relying on an exception handler, how the CPU carry flag propagates a single bit of information between two 16-bit additions to produce a correct 32-bit result, and why outputting a decimal number requires reversing the digit order obtained by dividing by 10.

The sign-magnitude representation chosen for signed numbers — storing a separate flag byte alongside the unsigned magnitude — trades a small amount of memory for significantly simpler arithmetic logic. It avoids the need to convert to and from two's-complement while still correctly handling negative inputs for addition and subtraction.

The program's limitations are equally instructive: the silent overflow on factorial inputs of 13 or greater, the 16-bit divisor ceiling, and the lack of floating-point support are all direct consequences of working within the constraints of a 16-bit real-mode environment without the safety nets that modern runtimes provide. These constraints make the program an honest and transparent illustration of what low-level computing actually involves.

Overall, this project serves as a solid foundation for anyone seeking to understand x86 assembly, DOS programming, or the mechanics of arithmetic at the hardware level. Further development could extend it with floating-point support, larger integer widths, or an expression parser, each of which would introduce additional layers of algorithmic and architectural thinking directly in assembly language.

— *End of Documentation* —